



comp10002/comp20005 program array1.c

Support material for Chapter 7 of
Programming, Problem Solving, and Abstraction with C,
by Alistair Moffat

(c) 2023, The University of Melbourne
Prepared by Alistair Moffat, ammoffat@unimelb.edu.au

```
#define N 5

int
main(int argc, char *argv[]) {

    int A[N], i;

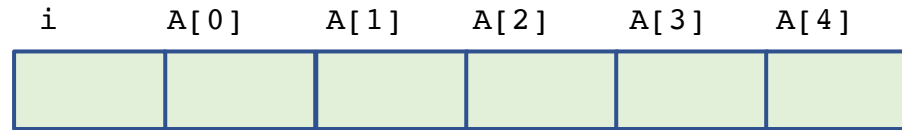
    for (i=0; i<N; i++) {
        A[i] = i*(i+1)/2;
    }

    printf("array has been filled\n");

    for (i=0; i<N; i++) {
        printf("A[%d]=%d\n", i, A[i]);
    }

    return 0;
}
```

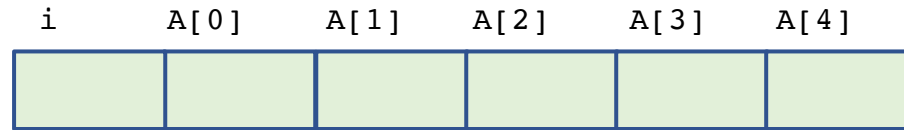
mac:



```
#define N 5
```

```
int  
main(int argc, char *argv[]) {  
  
    int A[N], i;  
  
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }  
  
    printf("array has been filled\n");  
  
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }  
  
    return 0;  
}
```

```
mac: ./array1
```



```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {
```

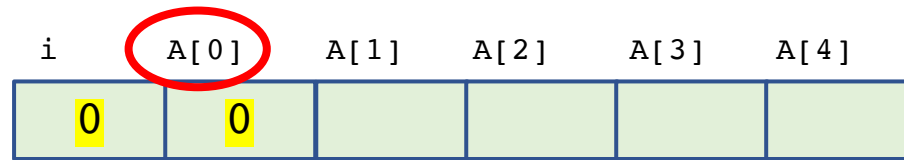
```
        printf("A[%d]=%d\n", i, A[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```



```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {
```

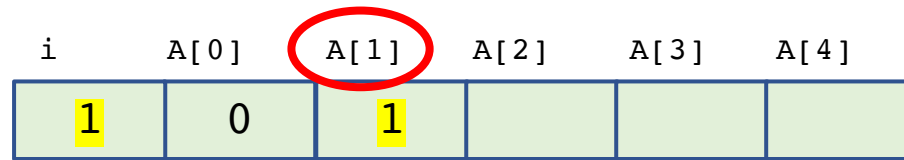
```
        printf("A[%d]=%d\n", i, A[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```



```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {
```

```
        printf("A[%d]=%d\n", i, A[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
2	0	1	3		

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {
```

```
        printf("A[%d]=%d\n", i, A[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
3	0	1	3	6	

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {
```

```
        printf("A[%d]=%d\n", i, A[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```


i	A[0]	A[1]	A[2]	A[3]	A[4]
4	0	1	3	6	10

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {
```

```
        printf("A[%d]=%d\n", i, A[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
5	0	1	3	6	10

```
#define N 5
```

```
int  
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
5	0	1	3	6	10

```
#define N 5

int
main(int argc, char *argv[]) {

    int A[N], i;

    for (i=0; i<N; i++) {
        A[i] = i*(i+1)/2;
    }

    printf("array has been filled\n");

    for (i=0; i<N; i++) {
        printf("A[%d]=%d\n", i, A[i]);
    }

    return 0;
}
```

```
mac: ./array1
array has been filled
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
0	0	1	3	6	10

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

```
A[0]=0
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
1	0	1	3	6	10

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

```
A[0]=0
```

```
A[1]=1
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
2	0	1	3	6	10

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

```
A[0]=0
```

```
A[1]=1
```

```
A[2]=3
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
3	0	1	3	6	10

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

```
A[0]=0
```

```
A[1]=1
```

```
A[2]=3
```

```
A[3]=6
```

i	A[0]	A[1]	A[2]	A[3]	A[4]
4	0	1	3	6	10

```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

```
A[0]=0
```

```
A[1]=1
```

```
A[2]=3
```

```
A[3]=6
```

```
A[4]=10
```


i	A[0]	A[1]	A[2]	A[3]	A[4]
5	0	1	3	6	10

```
#define N 5

int
main(int argc, char *argv[]) {

    int A[N], i;

    for (i=0; i<N; i++) {
        A[i] = i*(i+1)/2;
    }

    printf("array has been filled\n");

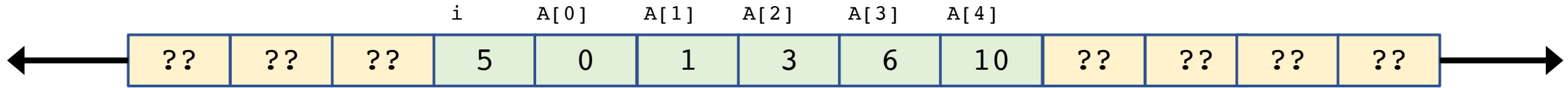
    for (i=0; i<N; i++) {
        printf("A[%d]=%d\n", i, A[i]);
    }

    return 0;
}
```

```
mac: ./array1
array has been filled
A[0]=0
A[1]=1
A[2]=3
A[3]=6
A[4]=10
mac:
```

so far, so good...

but memory is a continuous
sequence of words, and there
are no fences...



```
#define N 5
```

```
int  
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

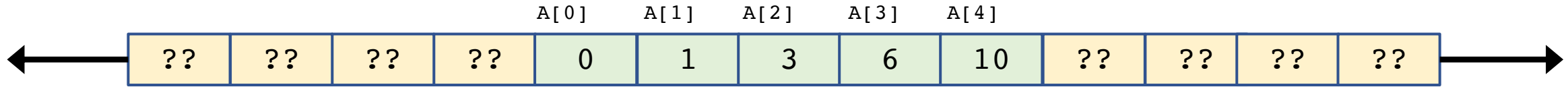
```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```



```
#define N 5
```

```
int  
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

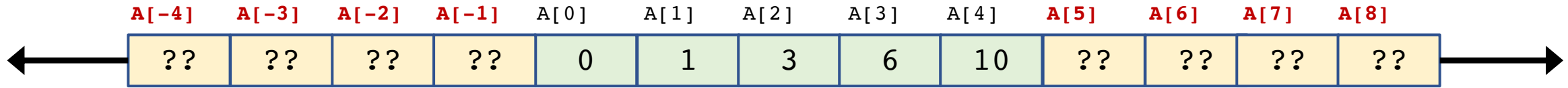
```
    printf("array has been filled\n");
```

```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1  
array has been filled
```



```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

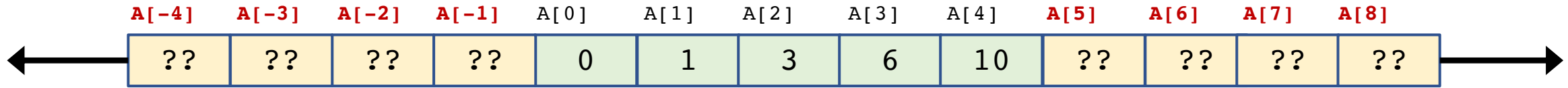
```
    for (i=0; i<N; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```



```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {
```

```
        A[i] = i*(i+1)/2;
```

```
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=-4; i<9; i++) {
```

```
        printf("A[%d]=%d\n", i, A[i]);
```

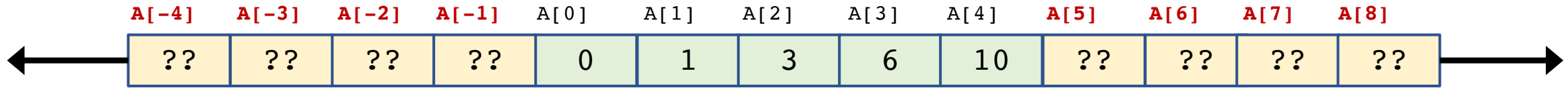
```
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```



```
#define N 5
```

```
int  
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

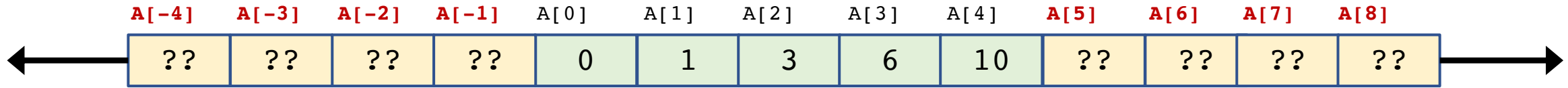
```
    printf("array has been filled\n");
```

```
    for (i=-4; i<9; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1  
array has been filled
```

```
#define N 5

int
main(int argc, char *argv[]) {

    int A[N], i;

    for (i=0; i<N; i++) {
        A[i] = i*(i+1)/2;
    }

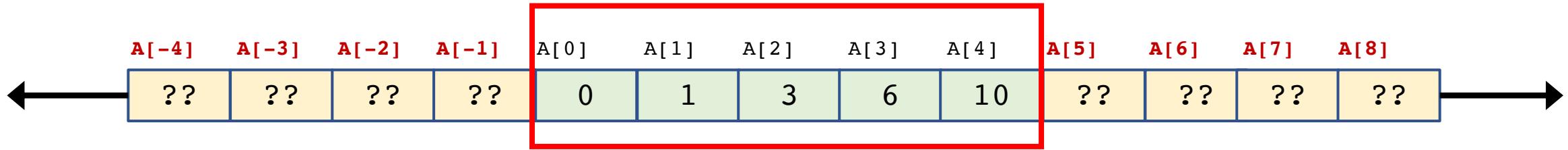
    printf("array has been filled\n");

    for (i=-4; i<9; i++) {
        printf("A[%d]=%d\n", i, A[i]);
    }

    return 0;
}
```

```
mac: ./array1
array has been filled
A[-4]=-362145392
A[-3]=32766
A[-2]=1
A[-1]=0
A[0]=0
A[1]=1
A[2]=3
A[3]=6
A[4]=10
A[5]=32766
A[6]=752156827
A[7]=1211439137
A[8]=-362145408
mac:
```





```
#define N 5
```

```
int
```

```
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=0; i<N; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=-4; i<9; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

```
mac: ./array1
```

```
array has been filled
```

```
A[-4]=-362145392
```

```
A[-3]=32766
```

```
A[-2]=1
```

```
A[-1]=0
```

```
A[0]=0
```

```
A[1]=1
```

```
A[2]=3
```

```
A[3]=6
```

```
A[4]=10
```

```
A[5]=32766
```

```
A[6]=752156827
```

```
A[7]=1211439137
```

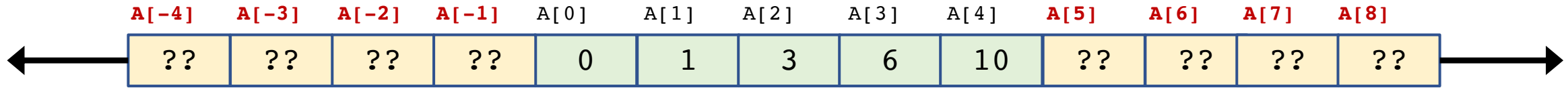
```
A[8]=-362145408
```

```
mac:
```

!

!

hmmm, that's a bit weird...



```
#define N 5
```

```
int  
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

```
    for (i=-4; i<9; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

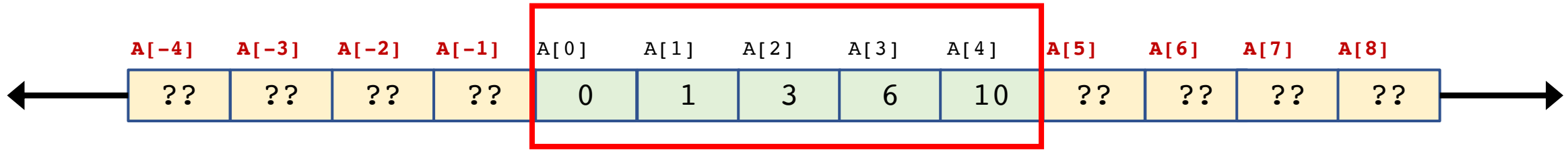
```
    printf("array has been filled\n");
```

```
    for (i=-4; i<9; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```

?!?



```
#define N 5
```

```
int  
main(int argc, char *argv[]) {
```

```
    int A[N], i;
```

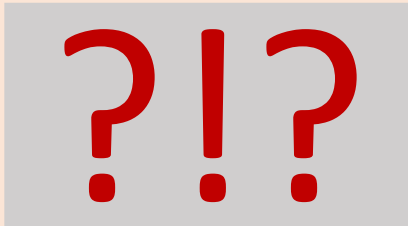
```
    for (i=-4; i<9; i++) {  
        A[i] = i*(i+1)/2;  
    }
```

```
    printf("array has been filled\n");
```

```
    for (i=-4; i<9; i++) {  
        printf("A[%d]=%d\n", i, A[i]);  
    }
```

```
    return 0;
```

```
}
```



```
mac: ./array1
```

```
array has been filled
```

```
A[-4]=6
```

```
A[-3]=3
```

```
A[-2]=1
```

```
A[-1]=0
```

```
A[0]=0
```

```
A[1]=1
```

```
A[2]=3
```

```
A[3]=6
```

```
A[4]=10
```

```
A[5]=15
```

```
A[6]=21
```

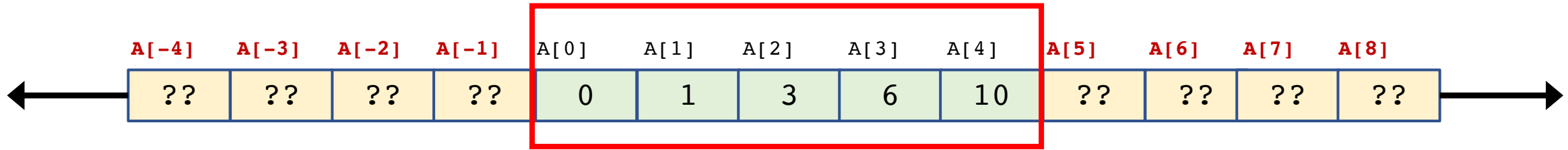
```
A[7]=28
```

```
A[8]=36
```

```
Abort
```

```
mac:
```





```
#define N 5

int
main(int argc, char *argv[]) {

    int A[N], i;

    for (i=-4; i<9; i++) {
        A[i] = i*(i+1)/2;
    }

    printf("array has been filled\n");

    for (i=-4; i<9; i++) {
        printf("A[%d]=%d\n", i, A[i]);
    }

    return 0;
}
```

```
mac: ./array1
array has been filled
A[-4]=6
A[-3]=3
A[-2]=1
A[-1]=0
A[0]=0
A[1]=1
A[2]=3
A[3]=6
A[4]=10
A[5]=15
A[6]=21
A[7]=28
A[8]=36
Abort
mac:
```

phew!

this time I got lucky:
the program gave a message

this time I got lucky:
the program gave a message

other times it might continue on,
but with **corrupted variables** ...

this time I got lucky:
the program gave a message

other times it might continue on,
but with corrupted variables ...

... and your programming mistake
could lead to **loss** or **death**